15) Lecture 26-27 | String Matching | KMP Algorithm |

03 October 2026 22:47

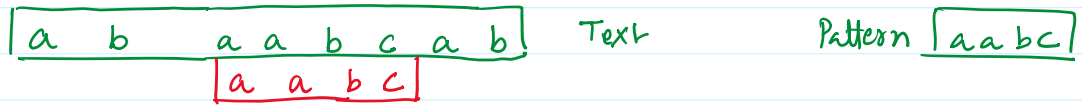
Input two strings text and pattern

Output Finding occurance of pattern in text

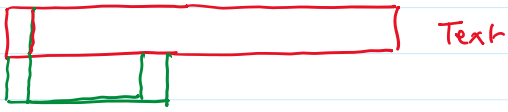
1st occurance of pattern in text

Notation

m := size of pattern
n := size of text



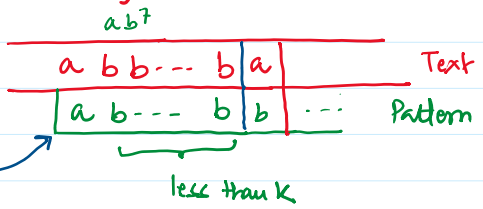
Naive algo: ∀ index (over text) i from 1 to n
check if position i onwards pattern exist



Running time $O(mn)$.

Ex: Pattern := $ab\underbrace{b\cdots b}_{k\text{ times}} = ab^k$

$\Sigma = \{a, b\}$



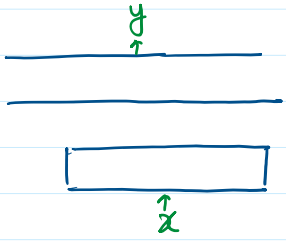
- No point of shifting the pattern by 1
- somehow utilize the information of match



↑ move pattern at position of mismatch

ALG (For this example only)

while ($y \leq n - m + 1$)



If $P[x] = T[y]$, then

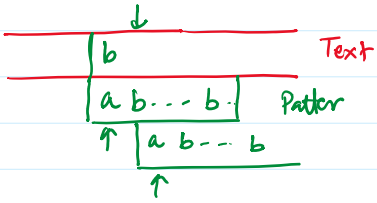
- $x \leftarrow x+1$
- $y \leftarrow y+1$

/* Match

else

- if ($x \neq 1$) $x \leftarrow 1$
- if ($x = 1$) $y \leftarrow y+1$

/* Mismatch



Report match ($x == m+1$) /* Pattern is found

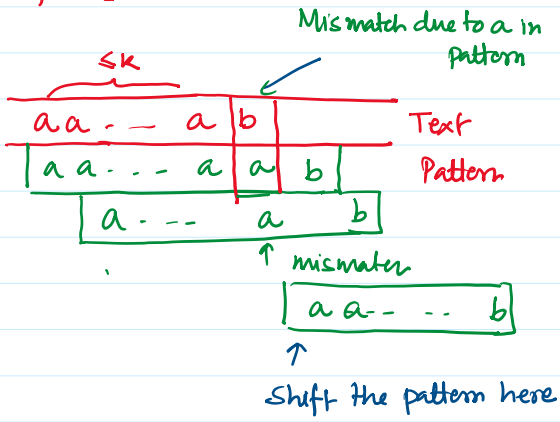
Time: - For every matched position in text exactly one comparision
- " mismatch " " two "

For pattern ab^k Running time $\leq 2n = \theta(n)$

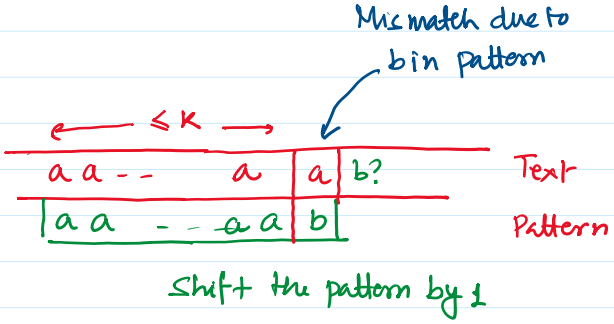
Ex:2 Pattern $a^k b :=$ a . . a b (a repeated k times)

$\Sigma = \{a, b\}$

2.a



2.b



Only for pattern $a^k b$

Shift the pattern by 1

If there is match we move both text and pattern pointer together

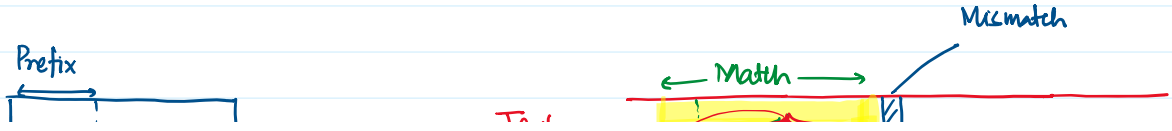
- If there is a mismatch due to "a" (in pattern)
  move the pattern till the position of mismatch

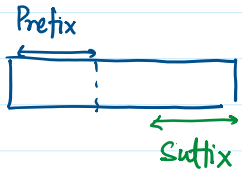
- If there is a mismatch due to "b" (in pattern)
  move the pattern by 1

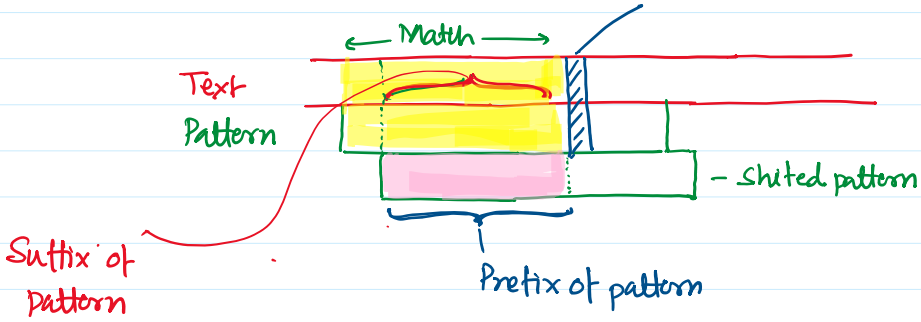
Time: - For every matched position in text exactly one comparision $= \theta(n)$
- " mismatch " " two "

For Ex 1 and Ex 2 we got O(n) algo

Que Can we get O(n) algo for all pattern?

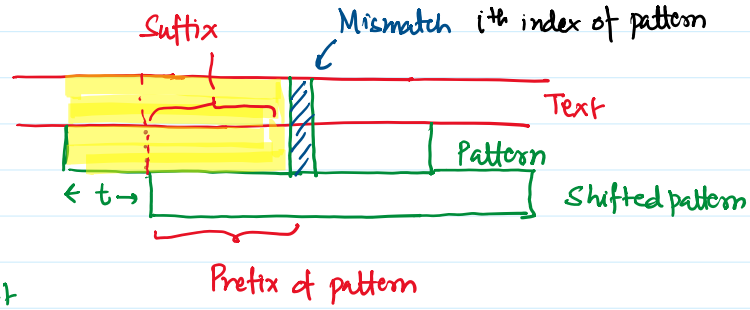




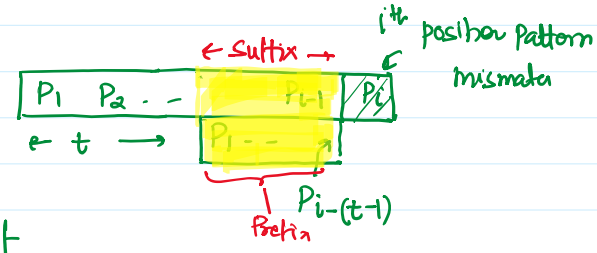


- When mismatch happen how much to shift is the property of pattern

- Prefix of shifted pattern should match with suffix, and it should be largest (prefix | suffix)



- Once mismatch happen how much to shift depends on pattern



Should shift the pattern by t index if

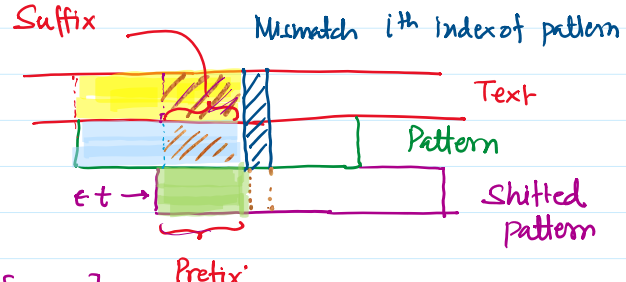
$P_1\ P_2\ --\ P_{i-(t-1)}$ is suffix of $P_1\ --\ P_{i-1}$

$P_1\ P_2\ -\ P_{(i-j)}$ is not $P_1\ --\ P_{i-1}$ for $j < t-1$

This is function of pattern and can be pre computed.

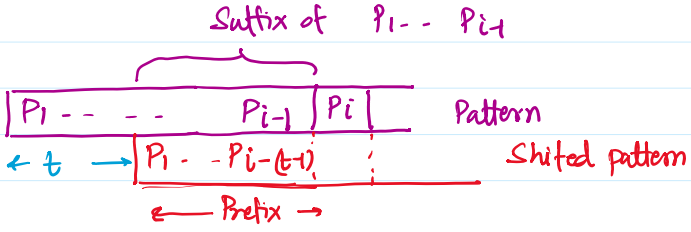
{ For each position of pattern find the largest prefix which is also its suffix (pattern chopped until position i)

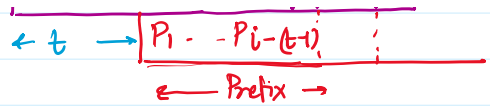
Preprocessing step $O(m)$



We need to compute "the amount of shift" for pattern chopped until index i $\forall i \in [0, n-1]$

We want shift amount "t" to be as large as possible

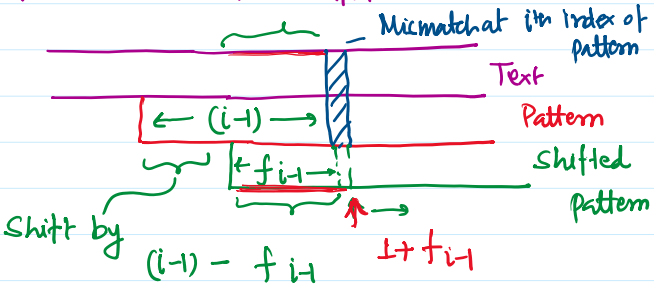




Should shift the pattern by t unit
if $P_1 \dots P_{i-(t-1)}$ is suffix of $P_1 \dots P_{i-1}$
$P_1 \dots P_{i-j}$ is not suffix of $P_1 \dots P_{i-1}$ for $j < (t-1)$

f_{i-1} := length of longest prefix of $P_1 \dots P_{i-1}$ that is also its suffix

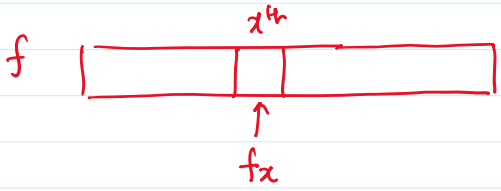
Once mismatch happens due to index i of pattern, then we shift the pattern by $(i-1) - f_{i-1}$



Suppose we know how to compute f_i $\forall i = \{1 \dots m\}$, and we store it in an array.

ALG Pointer to pattern x P[x] : Value at x^{th} index of patter
" text y T[y] " y^{th} " text

$x \leftarrow 1$, $y \leftarrow 1$



while loop $(y \leq n-m+1)$

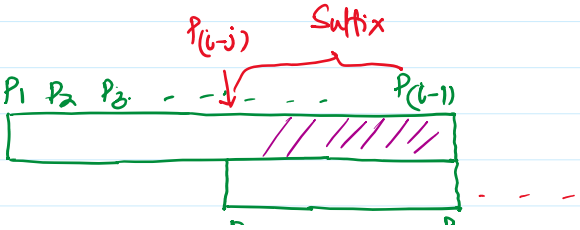
- if $P[x] == T[y]$
  - $x \leftarrow x+1$
  - $y \leftarrow y+1$
- else
  - if $(x \neq 1)$, then $x \leftarrow f_{x-1}+1$
  - if $(x=1)$, then $y \leftarrow y+1$
- If $(x == m+1)$, then "Pattern is found"

Running Time Another argument: when mismatch happen, shift the pattern by 1 (right side)
of times, shift the pattern is $(n-1)$.

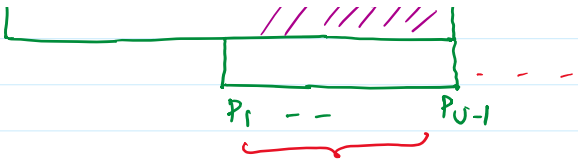
How to compute f_i for pattern

Naive Algo

For each $(i,j) \in [m] \times [m]$ Shift the



For each $(i,j) \in [m] \times [m]$ Shift the pattern by j position and check

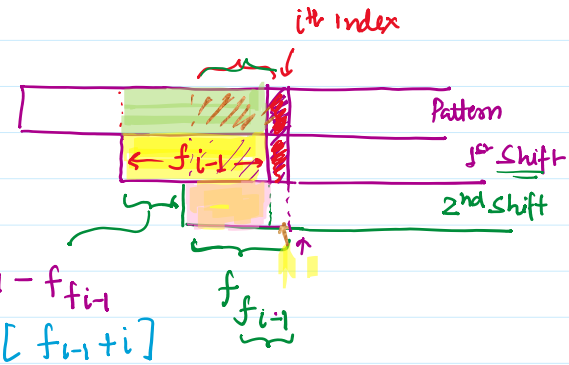


if $P_1 \text{ -- } P_{j-1}$ matches with $P_{(i-j)} \text{ ... } P_{(i-1)}$

Time $O(m^2)$

Qne: Can we compute f_j in $O(m)$ time?

- We will use inductive approach
- Suppose we have computed $f_1, f_2, \text{ -- } f_{i-1}$



We will see how to compute f_i

If ith position matches $\quad P[i] = P[f_{i-1}+i]$

$f_i \leftarrow f_{i-1} + 1$

If ith position does not match

Shift the pattern by

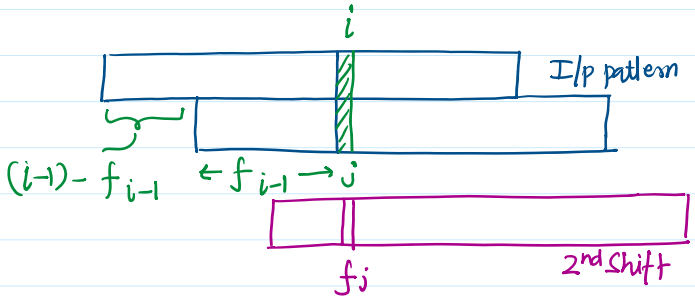
$$f_{i-1} - f_{f_{i-1}}$$

[index f_{i-1} is smaller than $(i-1)$ which we have already computed

[Need to compute (longest) prefix of $P_1 \text{ -- } P_i$ that matches with its suffix

Position of input pattern i

" Shifted " j



$f_0 \leftarrow 0 \qquad i \leftarrow 2 \qquad j \leftarrow 1$

while (loop until $i \leq m$)

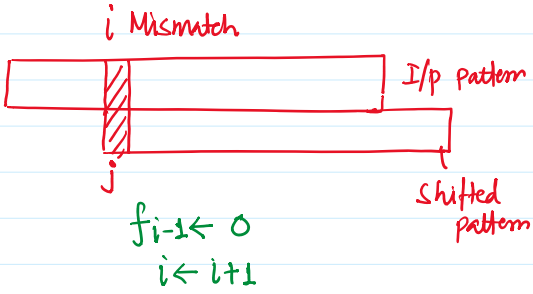
[If $P[i] == P[j]$, then

[$f_{i-1} \leftarrow j$, $i \leftarrow i+1$, $j \leftarrow j+1$

else

[if $(j \neq 1)$, then $\quad j \leftarrow f_j + 1$

[if $(j = 1)$, then $\quad f_{i-1} \leftarrow 0$; $i \leftarrow i+1$



L

Running time = (# if clause) + (# else clause)

= # times i incremented + # of time (i-j) incremented

$\leq m + m = 2m = O(n)$